

Modern C++

Trainer: Nilesh Ghule

constexpr: Compile-Time Constants

- constexpr (C++11) indicates a variable or function can participate in constant expressions
- const means not modifiable; constexpr requires compile-time initialization
- A constexpr variable is also const, but not every const is constexpr
- Use const for runtime-initialized non-modifiable values; use constexpr for true compile-time constants

```
const int x = 10;
constexpr int y = 20;

int input;
std::cin >> input;

const int a = input;     // Valid: runtime initialization
constexpr int b = input; // Error: input is not a constant expression
```

constexpr Functions and static_assert

- constexpr functions can be evaluated at compile time when required
- They can also be called at runtime with runtime arguments
- static_assert checks a condition during compilation
- Use static_assert for compile-time assumptions.

```
constexpr int square(int n) { return n * n; }

constexpr int result = square(5);
static_assert(result == 25);

int value;
std::cin >> value;
int runtimeResult = square(value); // evaluated at runtime

constexpr int BufferSize = 1024;
static_assert(BufferSize > 0, "Buffer size must be positive");
```

Compiler err if cond is false

Nilesh Ghule

constexpr Evolution and Common Mistakes

- C++11: introduced with strict function-body restrictions
  - Function body generally allowed only a single return statement.
- C++14: relaxed bodies (local variables, if, loops, conditionals)
  - Allows mutation of local variables in compile time evalations
- C++17: more constant-expression capabilities, if constexpr
- C++20/23: further expansion
- Mistakes: assuming const means constexpr; assuming every constexpr call runs at compile time

Nilesh Ghule

consteval in Modern C++ (C++20)

- Compile-Time Evaluation — Guaranteed
- Declares an immediate function that must be evaluated at compile time.
- If the call cannot be evaluated at compile time, the compiler reports an error.
- Use consteval when a computation must happen at compile time, ensuring errors are caught during compilation.
  - Forcing Compile-Time Initialization of Writable (Non-Const) Variables
  - Preventing Unintended Runtime Overhead for Heavy Helpers
  - Foundation for Compile-Time Reflection & Metaprogramming (C++26)

```
consteval int square(int x) {
    return x * x;
}

int a = square(5);  // OK: evaluated at compile time

int n = 5;
// int b = square(n); // Error: n isn't a const expr
```

Nilesh Ghule

const vs constexpr vs consteval

Feature	const	constexpr	consteval
Introduced	C++98	C++11	C++20
Meaning	Cannot modify after initialization	Can be evaluated at compile time	Must be evaluated at compile time
Compile-time evaluation	Not guaranteed	When possible/required	Mandatory for calls
Runtime use	Yes	Yes	No, for calls
Functions	Not applicable globally	Allowed	Allowed
Typical use	Read-only variables	Compile-time constants & flexible functions	Enforce compile-time computation

Nilesh Ghule

inline: ODR and Header Definitions

- A program is divided into translation units; the One Definition Rule (ODR) restricts definitions
- Defining a normal function in a header included by multiple .cpp files can cause multiple-definition errors
- inline (language meaning) permits equivalent definitions in multiple translation units
- Traditional solution: declaration in header, definition in .cpp file

```
// MathUtils.h
#pragma once
inline int square(int n) {
    return n * n;
}

// Without inline, include in multiple .cpp files causes linker error.
```

Nilesh Ghule

inline Does Not Force Machine-Code Inlining

- inline is not a performance guarantee; compilers decide inlining independently
- A function can be inlined without inline keyword; inline function may not be inlined → fully compiler decision : replacement.
- Use inline for header-defined functions when ODR rules require it
- Do not add inline merely because a function is short

```
inline int calculateTotal(int price, int quantity) {
    return price * quantity;
}
// Compiler may or may not inline this call.
```

inline variables

- Traditionally global variables are defined in .cpp file and declare in .h file using extern keyword.
- If variable declared in header and used in multiple files, will cause linker error.
- C++17: inline can be applied to variables.
- myfile.h
  inline int var = 1;
- myfile.h can now be included in multiple .cpp files without linker error. Linker ensure to have single copy of the variable in the scope.
- inline variables in class scope should be static.

```
class Config {
public:
    inline static int count = 0;
};
```

Nilesh Ghule

inline constexpr Variables for Shared Constants

- inline constexpr variable can be defined in a header and included by multiple .cpp files
- No separate .cpp definition needed for these constants
- inline here is about linkage/ODR, not machine-code substitution

```
// AppConstants.h
#pragma once
namespace app {
    inline constexpr int DefaultPort = 8080;
    inline constexpr int MaxRetries = 3;
}
```

- Advantages:
  - constexpr — compile-time constant.
  - inline (C++17) — allows definition in a header included by multiple translation units.
  - Scope — namespace or class members avoid global macro pollution.
- Class scoped static *constexpr* variables are by default *inline*.

Nilesh Ghule

Scoped Enumerations: enum class

- An enumeration represents a finite set of named values
- Ordinary enum: enumerator names leak into surrounding scope; implicit conversion to int is allowed
- enum class (C++11): enumerator names are scoped; no implicit conversion to int
- The type is distinct from an integer—safer for representing states

```
enum class Status { ready, running, failed };

Status current = Status::running;

if (current == Status::failed) {
    // Handle failure.
}

// int code = current; // Error: no implicit conversion
int code = static_cast<int>(current); // Explicit conversion
```

Nilesh Ghule

enum class: Underlying Type and switch

- Default underlying type is int; can specify another integral type when needed
- Use enum class in switch statements for clear, scoped case labels
- Each non-empty case normally needs break; revise switch cases when enum changes.

```
enum class Status { ready, running, failed };

switch (current) {
    case Status::ready:
        // Start
        break;
    case Status::running:
        // Report progress
        break;
    case Status::failed:
        // Report failure
        break;
}
```

```
enum class PacketType : char {
    data = 1,
    acknowledgement = 2,
    error = 3
};

switch (option) {
    using enum PacketType;
    case data: // …
        break;
    case acknowledgement: // …
        break;
    case error: // …
        break;
}
```

Nilesh Ghule

if and switch with Initializer (C++17)

- Allows an initialization statement before the condition of if or switch
- Keeps temporary variables in the narrowest necessary scope
- Syntax: if (init-statement; condition) and switch (init-statement; condition)
- Do not omit the semicolon between initializer and condition

```
if (const auto it = options.find("mode"); it != options.end())
    std::cout << it->second << '\n';
else
    std::cout << "Mode not specified\n";

switch (const auto state = Status::running; state) {
    case Status::ready:   /* ... */ break;
    case Status::running: /* ... */ break;
    case Status::failed:  /* ... */ break;
}
```

Nilesh Ghule

Initializers in Loops — C++20

- Keep loop setup and iteration logic together
- C++ allows an initializer in a for loop, and C++20 introduces an initializer in a range-based for loop.

```
for (int i = 0; i < 5; ++i) {
    // Use i
}
```

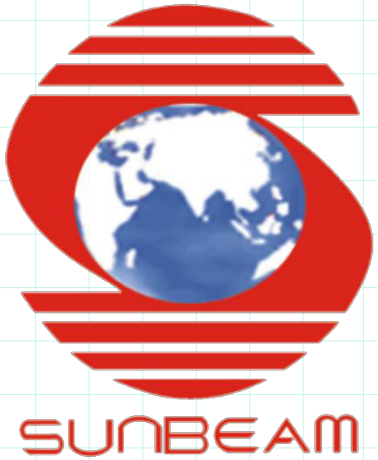
C++98

```
for (auto values = getValues();
     const auto& value : values) {
    // Use value
}
```

- Range-based for loop with initializer (C++20)
  - values is initialized once before iteration begins.
  - values remains in scope throughout the loop.
  - The initializer keeps the collection's lifetime and usage together.

- Use a range-based for initializer when a collection needs local setup before iteration, improving readability and scope management.

Nilesh Ghule



Thank You!

Nilesh Ghule

<nilesh@sunbeaminfo.com>